
PYTHON PROGRAMMING

ASSIGNMENT

Understanding and Implementation of Python Packages



DEPARTMENT OF COUMPUTER SCIENCE & ENGINEERING

ANIL NEERUKONDA INSRITUTE OF TECHNOLOGY AND SCIENCES

(2025-29)

NAME: J. DINESH KUMAR

SEC: CSE-A

ROLL NO: A25126510019

Python Standard Library Packages Guide

Concise reference with examples for 12 essential built-in packages

CPython 3.x Edition

#	Package	Primary Use
1	os	File system & process management
2	sys	Interpreter internals & CLI args
3	math	Mathematical functions & constants
4	random	Pseudo-random number generation
5	datetime	Date & time manipulation
6	collections	Specialised container data types
7	itertools	Memory-efficient iterator tools
8	functools	Higher-order functions & decorators
9	re	Regular-expression pattern matching
10	json	JSON serialisation / deserialisation
11	pathlib	Object-oriented filesystem paths
12	threading	Concurrent I/O-bound execution

Table of Contents

os

File & Directory Operations 4

Environment Variables 4

sys

Command-line Arguments & Exit 5

Module Search Path 5

math

Common Mathematical Functions 6

Trigonometry & Constants 6

random

Basic Random Operations 7

Seeding for Reproducibility 7

datetime

Date & Time Basics 8

Parsing & Formatting 8

collections

Counter & defaultdict 9

deque & namedtuple 9

itertools

Combinatorics 10

Infinite & Grouping Iterators 10

functools

lru_cache & partial 11

reduce & wraps 11

re

Search, Match & Findall 12

Groups, Sub & Split 12

json

Serialisation & Deserialisation 13

Reading & Writing Files 13

pathlib

Path Construction & Properties 14

Reading, Writing & Iterating 14

threading

Creating & Starting Threads 15

Lock — Shared-State Safety 15

os

The `os` module provides a portable way of using operating system–dependent functionality such as reading or writing to the file system, working with environment variables, and managing processes. It abstracts platform differences so the same code runs on Windows, macOS, and Linux.

File & Directory Operations

```
import os

# Current working directory
print(os.getcwd())

# List directory contents
for item in os.listdir('.'):
    print(item)

# Create and remove a directory
os.makedirs('demo/sub', exist_ok=True)
os.rmdir('demo/sub')
os.rmdir('demo')

# Check existence
print(os.path.exists('demo'))          # False
print(os.path.isfile('generate_pdf.py'))
```

Environment Variables

```
import os

# Read an env variable (with fallback)
home = os.environ.get('HOME', '/unknown')
print(f"Home: {home}")

# Set a variable for child processes
os.environ['MY_VAR'] = 'hello'
print(os.environ['MY_VAR'])
```

sys

The sys module exposes interpreter-level information and control: command-line arguments, the module search path, stdin/stdout/stderr streams, and hooks for exiting the program. It is indispensable for building command-line tools and inspecting runtime state.

Command-line Arguments & Exit

```
import sys

print("Python version:", sys.version)
print("Platform:", sys.platform)
print("Arguments:", sys.argv)          # ['script.py', ...]

# Graceful exit with a message
# sys.exit("Error: missing argument")
```

Module Search Path

```
import sys

# View the current search path
for path in sys.path:
    print(path)

# Add a custom path at runtime
sys.path.insert(0, '/tmp/my_libs')
```

math

The math module gives access to C-level mathematical functions defined by the C standard. It covers trigonometry, logarithms, factorials, combinatorics, and useful constants such as pi and e. All functions operate on real (float) numbers.

Common Mathematical Functions

```
import math

print(math.sqrt(144))      # 12.0
print(math.factorial(6))   # 720
print(math.gcd(48, 18))    # 6
print(math.log(1000, 10))  # 3.0
print(math.ceil(4.2))      # 5
print(math.floor(4.9))     # 4
```

Trigonometry & Constants

```
import math

angle = math.radians(45)
print(math.sin(angle))     # 0.7071...
print(math.cos(angle))     # 0.7071...
print(math.tan(angle))     # 1.0
print(math.pi)            # 3.14159...
print(math.e)              # 2.71828...
print(math.inf)            # inf
```

random

The random module implements pseudo-random number generators for various distributions. It is widely used in simulations, games, testing, and data sampling. For cryptographic use, prefer the secrets module instead.

Basic Random Operations

```
import random

print(random.random())           # float in [0.0, 1.0)
print(random.randint(1, 100))    # inclusive int
print(random.uniform(1.5, 9.5))  # float in range

items = ['apple', 'banana', 'cherry']
print(random.choice(items))      # one random element
random.shuffle(items)
print(items)
print(random.sample(items, 2))    # 2 unique elements
```

Seeding for Reproducibility

```
import random

random.seed(42)
print([random.randint(1, 10) for _ in range(5)])
# Always: same sequence for seed 42

# Weighted choice
options = ['A', 'B', 'C']
weights = [70, 20, 10]
result = random.choices(options, weights=weights, k=5)
print(result)
```

datetime

The datetime module supplies classes for manipulating dates and times in both simple and complex ways. Key classes are date, time, datetime, and timedelta. It also supports timezone-aware arithmetic through timezone.

Date & Time Basics

```
from datetime import date, time, datetime, timedelta

today = date.today()
print(today)                # 2025-05-01
print(today.strftime('%d %B %Y'))  # 01 May 2025

now = datetime.now()
print(now.isoformat())

delta = timedelta(days=30)
print(today + delta)        # 30 days later
```

Parsing & Formatting

```
from datetime import datetime

# Parse string to datetime
dt = datetime.strptime('15-08-2024 09:30', '%d-%m-%Y %H:%M')
print(dt.year, dt.month, dt.day)

# Format back to string
print(dt.strftime('%A, %B %d %Y'))  # Thursday, August 15 2024

# Difference between two datetimes
later = datetime(2025, 1, 1)
diff = later - dt
print(diff.days, "days")
```

collections

The collections module provides specialised container data types that are high-performance alternatives to built-in dicts, lists, and tuples. Notable members include Counter, defaultdict, OrderedDict, deque, and namedtuple.

Counter & defaultdict

```
from collections import Counter, defaultdict

# Counter – frequency map
words = ['cat', 'dog', 'cat', 'fish', 'dog', 'cat']
c = Counter(words)
print(c)                # Counter({'cat': 3, ...})
print(c.most_common(2)) # top 2

# defaultdict – auto-initialised keys
dd = defaultdict(list)
for k, v in [('a', 1), ('b', 2), ('a', 3)]:
    dd[k].append(v)
print(dict(dd))          # {'a': [1, 3], 'b': [2]}
```

deque & namedtuple

```
from collections import deque, namedtuple

# deque – O(1) appends/pops from both ends
dq = deque([1, 2, 3], maxlen=5)
dq.appendleft(0)
dq.append(4)
print(dq)

# namedtuple – lightweight immutable record
Point = namedtuple('Point', ['x', 'y'])
p = Point(3, 4)
print(p.x, p.y)
print(p._asdict())
```

itertools

The itertools module provides a collection of fast, memory-efficient tools for working with iterators. They can be combined to express complex data-processing pipelines in a readable, declarative style without materialising intermediate lists.

Combinatorics

```
import itertools

# All permutations of length 2
print(list(itertools.permutations('ABC', 2)))

# All combinations (order doesn't matter)
print(list(itertools.combinations([1,2,3,4], 2)))

# Cartesian product
print(list(itertools.product('AB', repeat=2)))
```

Infinite & Grouping Iterators

```
import itertools

# chain – flatten iterables
merged = list(itertools.chain([1,2], [3,4], [5]))
print(merged)           # [1, 2, 3, 4, 5]

# groupby – consecutive groups
data = [('A', 1), ('A', 2), ('B', 3), ('B', 4)]
for key, group in itertools.groupby(data, key=lambda x: x[0]):
    print(key, list(group))

# islice – lazy slicing of any iterable
counter = itertools.count(start=10, step=5)
print(list(itertools.islice(counter, 5))) # [10,15,20,25,30]
```

functools

The functools module offers higher-order functions and decorators that act on or return other functions. Highlights include lru_cache for memoisation, partial for currying, and reduce for fold operations.

lru_cache & partial

```
import functools

@functools.lru_cache(maxsize=128)
def fibonacci(n):
    if n < 2:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

print([fibonacci(i) for i in range(10)])

# partial – fix some arguments
def power(base, exp):
    return base ** exp

square = functools.partial(power, exp=2)
cube   = functools.partial(power, exp=3)
print(square(5), cube(3))    # 25, 27
```

reduce & wraps

```
import functools

# reduce – fold a sequence
product = functools.reduce(lambda acc, x: acc * x, [1,2,3,4,5])
print(product)    # 120

# wraps – preserve metadata of wrapped function
def decorator(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

@decorator
def greet(name):
    '''Say hello.'''
    return f"Hello, {name}!"

print(greet("World"))
print(greet.__name__, greet.__doc__)
```

re

The re module adds regular-expression matching operations similar to those found in Perl. Patterns are compiled into pattern objects that expose methods for searching, matching, splitting, and replacing within strings with great flexibility.

Search, Match & Findall

```
import re

text = "Order #4521 placed on 2024-08-15 for $99.99"

# Search – find first occurrence anywhere
m = re.search(r'\d{4}-\d{2}-\d{2}', text)
print(m.group())          # 2024-08-15

# Findall – all non-overlapping matches
prices = re.findall(r'\$\d{1,2}', text)
print(prices)             # ['$99.99']

# Match – only at string start
if re.match(r'Order', text):
    print("Starts with Order")
```

Groups, Sub & Split

```
import re

# Named groups
pattern = r'(?P<year>\d{4})-(?P<month>\d{2})-(?P<day>\d{2})'
m = re.search(pattern, '2024-08-15')
print(m.group('year'), m.group('month'))

# Sub – replace
cleaned = re.sub(r'\s+', ' ', 'too many spaces')
print(cleaned)

# Split on multiple delimiters
parts = re.split(r'[;,|]', 'a;b;c|d')
print(parts)          # ['a', 'b', 'c', 'd']
```

json

The json module encodes Python objects to JSON strings and decodes JSON strings back to Python objects. It is the standard library choice for working with REST APIs, configuration files, and data interchange between services.

Serialisation & Deserialisation

```
import json

data = {
    "name": "Alice",
    "age": 30,
    "skills": ["Python", "SQL"],
    "active": True
}

# Python dict -> JSON string
json_str = json.dumps(data, indent=2)
print(json_str)

# JSON string -> Python dict
parsed = json.loads(json_str)
print(parsed["skills"][0]) # Python
```

Reading & Writing Files

```
import json, pathlib

config = {"host": "localhost", "port": 5432, "debug": False}

# Write to file
with open('config.json', 'w') as f:
    json.dump(config, f, indent=2)

# Read from file
with open('config.json') as f:
    loaded = json.load(f)
print(loaded["port"]) # 5432

pathlib.Path('config.json').unlink() # cleanup
```

pathlib

The `pathlib` module offers an object-oriented API for filesystem paths. Path objects behave like smart strings: they support the `/` operator for joining, expose properties like `.stem` and `.suffix`, and include methods for reading, writing, and iterating over directory contents.

Path Construction & Properties

```
from pathlib import Path

p = Path('/home/user/docs/report.pdf')
print(p.name)      # report.pdf
print(p.stem)      # report
print(p.suffix)     # .pdf
print(p.parent)    # /home/user/docs

# Join paths with /
sub = Path('/tmp') / 'data' / 'output.csv'
print(sub)
```

Reading, Writing & Iterating

```
from pathlib import Path

tmp = Path('/tmp/demo_pathlib.txt')
tmp.write_text("Hello, pathlib!")
print(tmp.read_text())

# Iterate over files in a directory
for f in Path('/tmp').iterdir():
    if f.is_file():
        print(f.name)
        break

tmp.unlink()      # cleanup
```

threading

The threading module allows a Python program to run multiple threads concurrently within one process. Because of the Global Interpreter Lock (GIL), threads are most useful for I/O-bound tasks (network requests, file I/O) rather than CPU-bound computation.

Creating & Starting Threads

```
import threading, time

def worker(name, delay):
    time.sleep(delay)
    print(f"Thread {name} done after {delay}s")

threads = [
    threading.Thread(target=worker, args=(i, 1))
    for i in range(3)
]
for t in threads:
    t.start()
for t in threads:
    t.join()
print("All threads finished")
```

Lock — Shared-State Safety

```
import threading

counter = 0
lock = threading.Lock()

def increment(n):
    global counter
    for _ in range(n):
        with lock:
            counter += 1

threads = [threading.Thread(target=increment, args=(1000,))
           for _ in range(5)]
for t in threads: t.start()
for t in threads: t.join()
print("Counter:", counter)  # 5000
```